

Liquidity Modeling

When the music stops, in terms of liquidity, things will be complicated. But as long as the music is playing, you've got to get up and dance. We're still dancing.

—Chuck Prince (2007)

Liquidity is another important source of financial risk. However, it has been long neglected, and the finance industry has paid a huge price for modeling risk without considering liquidity.

The causes of liquidity risk are departures from the complete markets and symmetric information paradigm, which can lead to moral hazard and adverse selection. To the extent that such conditions persist, liquidity risk is endemic in the financial system and can cause a vicious link between funding and market liquidity, prompting systemic liquidity risk (Nikolaou 2009).

Tapping into the time lag between a changing value of the variable and its impact on the real market turns out to be a success criterion in modeling. For instance, interest rates, to some extent, diverge from real market dynamics from time to time, and it takes some time to settle. Together with this, uncertainty is the solely source of risk in traditional asset pricing models; however, it is far from reality. To fill the gap between financial models and real-market dynamics, the liquidity dimension stands out. A model with liquidity can better adjust itself to developments in the financial markets in that liquidity affects both the required returns of assets and also the level of uncertainty. Thus, liquidity is quite an important dimension in estimating probability of default (Gaygisiz, Karasan, and Hekimoglu 2021).

The importance of liquidity has been highlighted and has gained much attention since the global mortgage crisis broke out in 2007–2008. During this crisis, most financial institutions were hit hard by liquidity pressures, resulting in several strict measures taken by regulatory authorities and central banks. Since then, debates over

the need to include liquidity, originating from the lack of tradable securities, have intensified.

The concept of liquidity is multifaceted. By and large, a *liquid asset* is defined by the extent to which a large amount of it is sold without a considerable price impact. This is also known as *transaction cost*. This is, however, not the only important facet of liquidity. Rather, during a period of stress, resilience stands out as investors seek prompt price discovery (Sarr and Lybek 2002). This is pointed out by Kyle (1985): “Liquidity is a slippery and elusive concept, in part because it encompasses a number of transactional properties of markets.”

With that said, liquidity is an ambiguous concept, and to define it we need to focus on its different dimensions. In the literature, different researchers come up with different dimensions of liquidity, but for the purposes of this book, we will identify four defining characteristics of liquidity:

Tightness

The ability to trade an asset at the same price at the same time. This refers to the transaction cost occurring during a trade. If the transaction cost is high, the difference between buy and sell prices will be high or vice versa. So, a narrow transaction cost defines how tight the market is.

Immediacy

The speed at which a large amount of buy or sell orders can be traded. This dimension of liquidity provides us with valuable information about the financial market, as low immediacy refers to malfunctioning of parts of the market such as clearing, settlement, and so forth.

Depth

The presence of large numbers of buyers and sellers who are able to cover abundant orders at various prices.

Resiliency

A market's ability to bounce back from nonequilibrium. It can be thought of as a price-recovery process in which order imbalance dies out quickly.

Given the definition and interconnectedness of liquidity, it is not hard to see that modeling liquidity is a tough task. In the literature, many different types of liquidity models are proposed, however, considering the multidimensionality of liquidity, it may be wise to cluster the data depending on which dimension it captures. To this end, we will come up with different liquidity measures to represent all four dimensions. These liquidity measures are volume-based measures, transaction cost-based measures, price impact-based measures, and market-impact measures. For all these dimensions, several different liquidity proxies will be used.

Using clustering analysis, these liquidity measures will be clustered, which helps us to understand which part of liquidity an investor should focus on, because it is known that different dimensions of liquidity prevail in an economy during different time periods. Thus, once we are done with clustering analysis, we end up with a smaller number of liquidity measures. For the sake of clustering analysis, we will use the Gaussian mixture model (GMM) and the Gaussian mixture copula model (GMCM) to tackle this problem. GMM is a widely recognized clustering model that works well under elliptical distribution. GMCM is an extension of the GMM in that we include a copula analysis to take correlation into account. We will discuss these models in detail, so let us start by identifying the liquidity measures based on different liquidity dimensions.

Liquidity Measures

The role of liquidity has finally been recognized by finance practitioners and economists, which makes it even more important to understand and develop liquidity measurement. Existing literature concentrates on a single measure by which it is hard to conceptualize an elusive concept like liquidity. Instead, we will cover four dimensions to develop a more comprehensive application:

- Volume
- Transaction cost
- Price impact
- Market impact

Let's get started with the volume-based liquidity measures.

Volume-Based Liquidity Measures

Large orders are covered when the market has depth, that is, a deep financial market has the ability to meet abundant orders. This, in turn, provides information about the market, and if the market lacks depth, order imbalance and discontinuity emerge in the market. Given the market's depth, volume-based liquidity measures can be used to distinguish liquid and illiquid assets. Moreover, volume-based liquidity measures have a strong association with bid-ask spread: a large bid-ask spread implies low volume, while a narrow bid-ask spread implies high volume (Huong and Gregoriou 2020).

As you can imagine, a large portion of the variation in liquidity arises from trading activities. The importance of the volume-based approach is stressed by Blume, Easley, and O'Hara (1994) saying that volume traded generates information that cannot be extracted from alternative statistics.

To properly represent the depth dimension of liquidity, the following volume-based measures will be introduced:

- Liquidity ratio
- Hui-Heubel ratio
- Turnover ratio

Liquidity ratio

This ratio measures the extent to which volume is required to induce a price change of 1%:

$$LR_{it} = \frac{\sum_{t=1}^T P_{it} V_{it}}{\sum_{t=1}^T |PC_{it}|}$$

where P_{it} is the total price of stock i on day t , V_{it} represents the volume traded of stock i on day t , and finally, $|PC_{it}|$ is the absolute value of difference between price at time t and $t - 1$.

The higher the ratio LR_{it} is, the higher the liquidity of asset i will be. This implies that higher traded volume, $P_{it} V_{it}$, and low price difference, $|PC_{it}|$, amount to high liquidity level. Conversely, if a low volume is necessary to initiate a price change, then this asset is referred to as illiquid. Obviously, this conceptual framework focuses more on the price aspect than on the issue of time or on the execution costs typically present in a market (Gabrielsen, Marzo, and Zagaglia 2011).

Let's first import the data and observe it via the following codes. As it is readily observable, the main variables in the dataset are ask (ASKHI), bid (BIDLO), open (OPENPRC), and trading price (PRC) along with the volume (VOL), return (RET), volume-weighted return (vwretx) of the stock, and number of shares outstanding (SHROUT):

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")
plt.rcParams['figure.figsize'] = (10, 6)
pd.set_option('use_inf_as_na', True)

In [2]: liq_data = pd.read_csv('bid_ask.csv')

In [3]: liq_data.head()
```

```
(np.abs(liq_vol_all['PRC'][(i+1):(i+6)].mean() -
        liq_vol_all['PRC'][(i):(i+5)].mean()))))
```

❶ Calculating the required statistical measures for five-day window

Now, we have minimum bid price, max ask price, summation of volume traded, mean of the number of shares outstanding, and mean of the trading price per five days.

Hui-Heubel ratio

Another measure that captures the depth is the Hui-Heubel liquidity ratio, known as L_{HH} :

$$L_{HH} = \frac{P_{max} - P_{min}}{P_{min}} / V / \bar{P} \times \text{shrout}$$

where P_{max} and P_{min} show maximum and minimum price over the determined period, respectively. $\bar{P}$ is average closing price over determined period. What we have in the numerator is the percentage change in the stock price, and the volume traded is divided by market capitalization, i.e., $\bar{P} \times \text{shrout}$ in the denominator. One of the most distinguish features of Hui-Heubel liquidity measure is that it is applicable to a single stock, not only portfolios.

As discussed by Gabrielsen, Marzo, and Zagaglia (2011), P_{max} and P_{min} can be replaced by bid-ask spread but due to low volatility in bid-ask spread, it tends to bias downward.

To compute the Hui-Heubel liquidity ratio, we first have the liquidity measures in a list, then we add all these measures into the dataframe to have all-encompassing data:

```
In [7]: Lhh = []

for j in liq_vol_all.TICKER.unique():
    for i in range(len(liq_vol_all[liq_vol_all.TICKER == j])):
        Lhh.append((liq_vol_all['PRC'][(i):(i+5)].max() -
                    liq_vol_all['PRC'][(i):(i+5)].min()) /
                    liq_vol_all['PRC'][(i):(i+5)].min() /
                    (liq_vol_all['VOL'][(i):(i+5)].sum() /
                    liq_vol_all['SHROUT'][(i):(i+5)].mean() *
                    liq_vol_all['PRC'][(i):(i+5)].mean()))
```

Turnover ratio

Turnover ratio has long been treated as a proxy for liquidity. It is basically the ratio of volatility to number of shares outstanding:

$$LR_{it} = \frac{1}{D_{it}} \frac{\sum_{t=1}^T Vol_{it}}{\sum_{t=1}^T shrou_{it}}$$

where D_{it} denotes the number of trading days, Vol_{it} is the number of shares traded at time t , and $shrou_{it}$ shows the number of shares outstanding at time t . A large turnover rate indicates a high level of liquidity, in that turnover implies trading frequency. As turnover rate incorporates the number of shares outstanding, it makes it a more subtle measure of liquidity.

Turnover ratio is calculated based on daily data, and then all the volume-based liquidity measures are converted into a dataframe and are included in `liq_vol_all`:

```
In [8]: turnover_ratio = []

        for j in liq_vol_all.TICKER.unique():
            for i in range(len(liq_vol_all[liq_vol_all.TICKER == j])):
                turnover_ratio.append((1/liq_vol_all['VOL'].count() *
                                     (np.sum(liq_vol_all['VOL'][i:i+1]) /
                                      np.sum(liq_vol_all['SHROUT'][i:i+1])))

In [9]: liq_vol_all['liq_ratio'] = pd.DataFrame(liq_ratio)
        liq_vol_all['Lhh'] = pd.DataFrame(Lhh)
        liq_vol_all['turnover_ratio'] = pd.DataFrame(turnover_ratio)
```

Transaction Cost–Based Liquidity Measures

In the real world, buyers and sellers do not magically meet in a frictionless environment. Rather, intermediaries (brokers and dealers), equipment (computers and the like), patience (trades cannot be realized instantaneously), and a rule book that stipulates how orders are to be handled and turned into trades are required. Also, the orders of large, institutional investors are big enough to affect market prices. All of these imply the existence of trading costs, and just how to structure a market (and a broader marketplace) to contain these costs is a subtle and intricate challenge (Baker and Kıymaz (2013)). This led to the emergence of transaction cost.

Transaction cost is a cost an investor must bear during trade. It is referred to as any expenses related to the execution of trade. A distinction of transaction cost as explicit and implicit costs is possible. The former relates to order processing, taxes, and brokerage fees, while the latter includes more latent costs, such as bid-ask spread, timing of execution, and so on.

Transaction cost is related to the tightness and immediacy dimensions of liquidity. High transaction costs discourage investors to trade and this, in turn, decreases the number of buyers and sellers in the market so that the trading place diverges away from the more centralized market into a fragmented one, which result in a shallow market (Sarr and Lybek 2002). To the extent that transaction cost is low, investors are

willing to trade and this results in a flourished trading environment in which markets will be more centralized.

Similarly, an abundance of buyers and sellers in a low transaction cost environment refers to the fact that a large number of orders are traded in a short period of time. So, immediacy is the other dimension of liquidity, which is closely related to the transaction cost.

Even though there is an ongoing debate about the goodness of bid-ask spread as well as the assurance that these models provide, bid-ask spread is a widely recognized proxy for transaction cost. To the extent that bid-ask spread is a good analysis of transaction cost, it is also a good indicator of liquidity by which the ease of converting an asset into cash (or a cash equivalent) might be determined. Without going into further detail, bid-ask spread can be measured by quoted spread, effective spread, and realized spread methods. So at first glance, it may seem strange to calculate bid-ask spread, which can be easily calculated by these methods. But this is not the case in reality. When the trade cannot be realized inside the quotes, then the spread is no longer the observed spread on which these methods are based.

Percentage quoted and effective bid-ask spreads

The other two well-known bid-ask spreads are *percentage quoted* and *percentage effective* bid-ask spreads. Quoted spread measures the cost of completing a trade, that is, the difference in the bid-ask spread. There are different forms of quoted spread but for the sake of scaling, we'll choose the percentage quoted spread:

$$\text{Percentage quoted spread} = \frac{P_{ask} - P_{bid}}{P_{mid}}$$

where P_{ask} is the ask price of the stock and P_{bid} is the bid price of the stock.

The effective spread measures the deviation between trading price and the mid-price, which is sometimes called the true underlying value of the stock. When trades occur either within or outside the quotes, a better measure of trading costs is the percentage effective half spread, which is based on the actual trade price, and is computed on a percentage basis (Bessembinder and Venkataraman 2010):

$$\text{Effective spread} = \frac{2|P_t - P_{mid}|}{P_{mid}}$$

where P_t is the trading price of the stock and P_{mid} is the midpoint of the bid-ask offer prevailing at the time of the trade.

It is relatively easy to calculate the percentage quoted and effective bid-ask spreads, as shown in the following:

```
In [10]: liq_vol_all['mid_price'] = (liq_vol_all.ASKHI + liq_vol_all.BIDLO) / 2
        liq_vol_all['percent_quoted_ba'] = (liq_vol_all.ASKHI -
                                            liq_vol_all.BIDLO) / \
                                            liq_vol_all.mid_price
        liq_vol_all['percent_effective_ba'] = 2 * abs((liq_vol_all.PRC -
                                                    liq_vol_all.mid_price)) / \
                                                    liq_vol_all.mid_price
```

Roll's spread estimate

One of the first and foremost spread measures was proposed by Roll (1984). The *Roll spread* can be defined as:

$$\text{Roll} = \sqrt{-\text{cov}(\Delta p_t, \Delta p_{t-1})}$$

where Δp_t and Δp_{t-1} are the price differences at time t and at time $t - 1$, and cov denotes the covariance between these price differences.

Assuming that the market is efficient¹ and the probability of distribution of observed price changes is stationary, Roll's spread is motivated by the fact that serial correlation of price changes is a good proxy for liquidity.

One of the most important things to note in calculating Roll's spread is that positive covariance is not well-defined, and it consists of almost half of the cases. The literature puts forth several methods to remedy this shortcoming, and we'll embrace Harris's (1990) approach in the following:

```
In [11]: liq_vol_all['price_diff'] = liq_vol_all.groupby('TICKER')['PRC']\
        .apply(lambda x:x.diff())
        liq_vol_all.dropna(inplace=True)
        roll = []

        for j in liq_vol_all.TICKER.unique():
            for i in range(len(liq_vol_all[liq_vol_all.TICKER == j])):
                roll_cov = np.cov(liq_vol_all['price_diff'][i:i+5],
                                liq_vol_all['price_diff'][i+1:i+6]) ❶
                if roll_cov[0,1] < 0: ❷
                    roll.append(2 * np.sqrt(-roll_cov[0, 1]))
                else:
                    roll.append(2 * np.sqrt(np.abs(roll_cov[0, 1]))) ❸
```

¹ *Efficient market* refers to how well and fast current prices reflect all available information about the value of the underlying asset(s).

- ❶ Calculating the covariance between price differences for the five-day window
- ❷ Checking the case where the covariance is negative
- ❸ In the case of positive covariance, Harris's approach is applied

The Corwin-Schultz spread

The *Corwin-Schultz spread* is rather intuitive and easy to apply. It rests mainly on the following assumption: given that the daily high and low prices are typically buyer and seller initiated, respectively, the observed price change can be split into effective price volatility and bid-ask spread. So the ratio of high-to-low prices for a day reflects both the stock's variance and its bid-ask spread (Corwin and Schultz 2012; Abdi and Ranaldo 2017).

This spread proposes an entirely new approach based on the daily high and low prices only, and the logic behind it is summarized by Corwin and Schultz (2012) as “the sum of the price ranges over 2 consecutive single days reflect 2 days' volatility and twice the spread, while the price range over one 2-day period reflects 2 days' volatility and one spread”:

$$S = \frac{2(e^\alpha - 1)}{1 + e^\alpha}$$

$$\alpha = \frac{\sqrt{2\beta} - \sqrt{\beta}}{3 - 2\sqrt{2}} - \sqrt{\frac{\gamma}{3 - 2\sqrt{2}}}$$

$$\beta = \mathbb{E} \left(\sum_{j=0}^1 \left[\ln \left(\frac{H_{t+j}^0}{L_{t+j}^0} \right) \right]^2 \right)$$

$$\gamma = \mathbb{E} \left(\sum_{j=0}^1 \left[\ln \left(\frac{H_{t+1}^0}{L_{t+1}^0} \right) \right]^2 \right)$$

where H_t^A (L_t^A) denotes actual high (low) prices on day t and H_t^o or L_t^o the observed high (low) stock price on day t .

The Corwin-Schultz spread requires multiple steps to calculate, as it includes many variables. The following code presents our way of doing this calculation:

```

In [12]: gamma = []

        for j in liq_vol_all.TICKER.unique():
            for i in range(len(liq_vol_all[liq_vol_all.TICKER == j])):
                gamma.append((max(liq_vol_all['ASKHI'].iloc[i+1],
                                   liq_vol_all['ASKHI'].iloc[i] -
                                   min(liq_vol_all['BIDLO'].iloc[i+1],
                                       liq_vol_all['BIDLO'].iloc[i])) ** 2)
                               gamma_array = np.array(gamma)

In [13]: beta = []

        for j in liq_vol_all.TICKER.unique():
            for i in range(len(liq_vol_all[liq_vol_all.TICKER == j])):
                beta.append((liq_vol_all['ASKHI'].iloc[i+1] -
                             liq_vol_all['BIDLO'].iloc[i+1]) ** 2 +
                             (liq_vol_all['ASKHI'].iloc[i] -
                              liq_vol_all['BIDLO'].iloc[i]) ** 2)
                beta_array = np.array(beta)

In [14]: alpha = ((np.sqrt(2 * beta_array) - np.sqrt(beta_array)) /
                   (3 - (2 * np.sqrt(2)))) - np.sqrt(gamma_array /
                                                         (3 - (2 * np.sqrt(2))))
        CS_spread = (2 * np.exp(alpha - 1)) / (1 + np.exp(alpha))

In [15]: liq_vol_all = liq_vol_all.reset_index()
        liq_vol_all['roll'] = pd.DataFrame(roll)
        liq_vol_all['CS_spread'] = pd.DataFrame(CS_spread)

```

Price Impact–Based Liquidity Measures

In this section, we will introduce price impact–based liquidity measures by which we are able to gauge the extent to which price is sensitive to volume and turnover ratio. Recall that resiliency refers to the market responsiveness about new orders. If the market is responsive to the new order—that is, a new order correct the imbalances in the market—then it is said to be resilient. Thus, given a change in volume and/or turnover ratio, high price adjustment amounts to resiliency or vice versa.

We have three price impact–based liquidity measures to discuss:

- The Amihud illiquidity measure
- The Florackis, Andros, and Alexandros (2011) price impact ratio
- Coefficient of elasticity of trading (CET)

Amihud illiquidity

This liquidity proxy is a celebrated and widely recognized measure. Amihud illiquidity (2002) basically measures the sensitivity of the return to trading volume. More concretely, it gives us a sense about a change in absolute return as trading volume changes by \$1. The Amihud illiquidity measure, or ILLIQ for short, is well known among academics and practitioners:

$$ILLIQ = \frac{1}{D_{it}} \sum_{d=1}^{D_{it}} \frac{|R_{itd}|}{V_{itd}}$$

where R_{itd} is the stock return on day t at month t , V_{itd} represents the dollar volume on day d at month t , and D is the number of observation days in month t .

The Amihud measure has two advantages over many other liquidity measures. First, the Amihud measure has a simple construction that uses the absolute value of the daily return-to-volume ratio to capture price impact. Second, the measure has a strong positive relation with expected stock return (Lou and Tao 2017).

The Amihud illiquidity measure is not hard to calculate. However, before directly calculating the Amihud's measure, the dollar volume of stocks needs to be computed:

```
In [16]: dvol = []

for j in liq_vol_all.TICKER.unique():
    for i in range(len(liq_vol_all[liq_vol_all.TICKER == j])):
        dvol.append((liq_vol_all['PRC'][i:i+5] *
                     liq_vol_all['VOL'][i:i+5]).sum())
liq_vol_all['dvol'] = pd.DataFrame(dvol)

In [17]: amihud = []

for j in liq_vol_all.TICKER.unique():
    for i in range(len(liq_vol_all[liq_vol_all.TICKER == j])):
        amihud.append((1 / liq_vol_all['RET'].count()) *
                       (np.sum(np.abs(liq_vol_all['RET'][i:i+1])) /
                        np.sum(liq_vol_all['dvol'][i:i+1])))
```

The price impact ratio

Florackis, Andros, and Alexandros (2011) aimed to improve the Amihud illiquidity ratio, and came up with a new liquidity measure, Return-to-Turnover (RtoTR). The disadvantages of Amihud's illiquidity measure are listed by authors as:

- It is not comparable across stocks with different market capitalizations.
 - It neglects the investor's holding horizon.
-

To deal with these drawbacks, Florackis, Andros, and Alexandros presented a new measure, RtoTR, that replaces the volume ratio of Amihud's model with turnover ratio so that the new measure is able to capture the trading frequency:

$$RtoTR = \frac{1}{D_{it}} \sum_{d=1}^{D_{it}} \frac{|R_{itd}|}{TR_{itd}}$$

where TR_{itd} is the monetary volume of stock i on day d at month t , and the rest of the components are the same as in Amihud's illiquidity measure.

The measure is as easy to calculate as Amihud's measure, and it has no size bias because it includes the turnover ratio for capturing the trading frequency. This also helps us to examine the price and size effect.

The calculation of the price impact ratio is provided below:

```
In [18]: florackis = []

for j in liq_vol_all.TICKER.unique():
    for i in range(len(liq_vol_all[liq_vol_all.TICKER == j])):
        florackis.append((1 / liq_vol_all['RET'].count()) *
                        (np.sum(np.abs(liq_vol_all['RET'][i:i+1]) /
                                liq_vol_all['turnover_ratio'][i:i+1])))
```

Coefficient of elasticity of trading

CET is a liquidity measure proposed to remedy the shortcomings of time-related liquidity measures such as number of trades and orders per unit of time. These measures are adopted to assess the extent to which market immediacy affects the liquidity level.

Market immediacy and CET go hand in hand as it measures the price elasticity of trading volume and if price is responsive (i.e., elastic) to the trading volume, that amounts to a greater level of market immediacy:

$$CET = \frac{\% \Delta V}{\% \Delta P}$$

where $\% \Delta V$ refers to change in trading volume and $\% \Delta P$ denotes a change in price.

The Python code of the CET formula is provided below. As a first part of this application, percentage difference in volume and price are calculated. Then all price impact-based liquidity measures are stored in the `liq_vol_all` dataframe:

```
In [19]: liq_vol_all['vol_diff_pct'] = liq_vol_all.groupby('TICKER')['VOL']\
        .apply(lambda x: x.diff()).pct_change() ❶
liq_vol_all['price_diff_pct'] = liq_vol_all.groupby('TICKER')['PRC']\
        .apply(lambda x: x.diff()).pct_change() ❷
```

```

In [20]: cet = []

        for j in liq_vol_all.TICKER.unique():
            for i in range(len(liq_vol_all[liq_vol_all.TICKER == j])):
                cet.append(np.sum(liq_vol_all['vol_diff_pct'][i:i+1])/
                           np.sum(liq_vol_all['price_diff_pct'][i:i+1]))

In [21]: liq_vol_all['amihud'] = pd.DataFrame(amihud)
        liq_vol_all['florackis'] = pd.DataFrame(flорackis)
        liq_vol_all['cet'] = pd.DataFrame(cet)

```

- ❶ Calculating the percentage volume difference
- ❷ Calculating the percentage price difference

Market Impact-Based Liquidity Measures

Identifying the source of information is a big deal in finance because an unknown source of information might mislead investors and lead to unintended consequences. A price surge, for instance, arising from the market does not provide the same information as one arising from an individual stock. With that being said, a new source of information should be identified in a way to capture price movement properly.

To accomplish this task, we use the capital asset pricing model (CAPM), by which we can distinguish systematic and unsystematic risk. The famous slope coefficient in CAPM indicates systematic risks, and the unsystematic risk is attributable to individual stocks as long as market risk is removed.

As it is referenced in Sarr and Lybek (2002), Hui-Heubel embraces the following approach:

$$R_i = \alpha + \beta R_m + u_i$$

where R_i is the daily return on i^{th} stock, and u_i is the idiosyncratic or unsystematic risk.

Once we estimate residuals, u_i , from the equation, it is regressed over the volatility, V_i , and the estimated coefficient of V_i gives the liquidity level of the related stock, also known as the idiosyncratic risk:

$$u_i^2 = \gamma_1 + \gamma_2 V_i + e_i$$

where u_i^2 denotes the squared residuals, V_i is the daily percentage change in trading volume, and e_i is the residual term.

Larger γ_2 implies larger price movements, and this gives us a sense about the liquidity of the stock. Conversely, the smaller γ_2 leads to smaller price movements, indicating higher liquidity levels. In code, we have:

```
In [22]: import statsmodels.api as sm

In [23]: liq_vol_all['VOL_pct_change'] = liq_vol_all.groupby('TICKER')['VOL']\
        .apply(lambda x: x.pct_change())
        liq_vol_all.dropna(subset=['VOL_pct_change'], inplace=True)
        liq_vol_all = liq_vol_all.reset_index()

In [24]: unsys_resid = []

for i in liq_vol_all.TICKER.unique():
    X1 = liq_vol_all[liq_vol_all['TICKER'] == i]['vwretx'] ❶
    y = liq_vol_all[liq_vol_all['TICKER'] == i]['RET'] ❷
    ols = sm.OLS(y, X1).fit() ❸
    unsys_resid.append(ols.resid) ❹
```

- ❶ Assigning volume-weighted returns of all tickers as the independent variable
- ❷ Assigning returns of all tickers as the dependent variable
- ❸ Running the linear regression model with the defined variables
- ❹ Storing the residuals coming from the linear regression as an unsystematic factor

And then we calculate the market impact-based liquidity ratio:

```
In [25]: market_impact = {}

for i, j in zip(liq_vol_all.TICKER.unique(),
               range(len(liq_vol_all['TICKER'].unique()))):
    X2 = liq_vol_all[liq_vol_all['TICKER'] == i]['VOL_pct_change'] ❶
    ols = sm.OLS(unsys_resid[j] ** 2, X2).fit()
    print('***' * 30)
    print(f'OLS Result for {i}')
    print(ols.summary())
    market_impact[j] = ols.resid ❷

*****
OLS Result for INTC

                        OLS Regression Results
=====
Dep. Variable:          y      R-squared (uncentered):          0.157
Model:                  OLS    Adj. R-squared (uncentered):      0.154
Method:                 Least Squares    F-statistic:          46.31
Date:                  Thu, 02 Dec 2021    Prob (F-statistic):      7.53e-11
```

```

Time:                15:33:38   Log-Likelihood:            1444.9
No. Observations:    249       AIC:                -2888.
Df Residuals:        248       BIC:                -2884.
Df Model:             1
Covariance Type:     nonrobust

```

```

=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
VOL_pct_change    0.0008      0.000        6.805      0.000        0.001        0.001
=====
Omnibus:                373.849   Durbin-Watson:                1.908
Prob(Omnibus):           0.000   Jarque-Bera (JB):            53506.774
Skew:                    7.228   Prob(JB):                     0.00
Kurtosis:                73.344   Cond. No.                     1.00
=====

```

Notes:

[1] R^2 is computed without centering (uncentered) since the model does not contain a constant.

[2] Standard Errors assume that the covariance matrix of the errors is correctly specified.

OLS Result for MSFT

OLS Regression Results

```

=====
Dep. Variable:          y   R-squared (uncentered):            0.044
Model:                  OLS   Adj. R-squared (uncentered):          0.040
Method:                 Least Squares   F-statistic:                11.45
Date:                   Thu, 02 Dec 2021   Prob (F-statistic):          0.000833
Time:                   15:33:38   Log-Likelihood:              1851.0
No. Observations:       249   AIC:                -3700.
Df Residuals:           248   BIC:                -3696.
Df Model:                1
Covariance Type:        nonrobust

```

```

=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
VOL_pct_change    9.641e-05  2.85e-05        3.383      0.001      4.03e-05      0.000
=====
Omnibus:                285.769   Durbin-Watson:                1.533
Prob(Omnibus):           0.000   Jarque-Bera (JB):            11207.666
Skew:                    4.937   Prob(JB):                     0.00
Kurtosis:                34.349   Cond. No.                     1.00
=====

```

Notes:

[1] R^2 is computed without centering (uncentered) since the model does not contain a constant.

[2] Standard Errors assume that the covariance matrix of the errors is correctly specified.

OLS Result for IBM

OLS Regression Results

=====						
Dep. Variable:	y	R-squared (uncentered):	0.134			
Model:	OLS	Adj. R-squared (uncentered):	0.130			
Method:	Least Squares	F-statistic:	38.36			
Date:	Thu, 02 Dec 2021	Prob (F-statistic):	2.43e-09			
Time:	15:33:38	Log-Likelihood:	1547.1			
No. Observations:	249	AIC:	-3092.			
Df Residuals:	248	BIC:	-3089.			
Df Model:	1					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

VOL_pct_change	0.0005	7.43e-05	6.193	0.000	0.000	0.001
=====						
Omnibus:	446.818	Durbin-Watson:	2.034			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	156387.719			
Skew:	9.835	Prob(JB):	0.00			
Kurtosis:	124.188	Cond. No.	1.00			
=====						

Notes:

[1] R^2 is computed without centering (uncentered) since the model does not contain a constant.

[2] Standard Errors assume that the covariance matrix of the errors is correctly specified.

- ❶ Assigning percentage change in volume of all tickers as the independent variable
- ❷ Market impact the residual of this linear regression

Then, we include the market impact in our dataframe and observe the summary statistics of all the liquidity measures that we've introduced so far:

```
In [26]: append1 = market_impact[0].append(market_impact[1])
         liq_vol_all['market_impact'] = append1.append(market_impact[2]) ❶

In [27]: cols = ['vol_diff_pct', 'price_diff_pct', 'price_diff',
                 'VOL_pct_change', 'dvol', 'mid_price']
         liq_measures_all = liq_vol_all.drop(liq_vol_all[cols], axis=1)\
         .iloc[:, -11:]
         liq_measures_all.dropna(inplace=True)
         liq_measures_all.describe().T

Out[27]:
```

	count	mean	std	min \
liq_ratio	738.0	7.368514e+10	2.569030e+11	8.065402e+08
Lhh	738.0	3.340167e-05	5.371681e-05	3.966368e-06
turnover_ratio	738.0	6.491127e-03	2.842668e-03	1.916371e-03
percent_quoted_ba	738.0	1.565276e-02	7.562850e-03	3.779877e-03
percent_effective_ba	738.0	8.334177e-03	7.100304e-03	0.000000e+00
roll	738.0	8.190794e-01	6.066821e-01	7.615773e-02

CS_spread	738.0	3.305464e-01	1.267434e-01	1.773438e-40
amihud	738.0	2.777021e-15	2.319450e-15	0.000000e+00
florackis	738.0	2.284291e-03	1.546181e-03	0.000000e+00
cet	738.0	-1.113583e+00	3.333932e+01	-4.575246e+02
market_impact	738.0	8.614680e-05	5.087547e-04	-1.596135e-03

	25%	50%	75%	max
liq_ratio	1.378496e+10	2.261858e+10	4.505784e+10	3.095986e+12
Lhh	1.694354e-05	2.368095e-05	3.558960e-05	5.824148e-04
turnover_ratio	4.897990e-03	5.764112e-03	7.423111e-03	2.542853e-02
percent_quoted_ba	1.041887e-02	1.379992e-02	1.878123e-02	5.545110e-02
percent_effective_ba	3.032785e-03	6.851479e-03	1.152485e-02	4.656669e-02
roll	4.574986e-01	6.975982e-01	1.011879e+00	4.178873e+00
CS_spread	2.444225e-01	3.609800e-01	4.188028e-01	5.877726e-01
amihud	1.117729e-15	2.220439e-15	3.766086e-15	1.320828e-14
florackis	1.059446e-03	2.013517e-03	3.324181e-03	7.869841e-03
cet	-1.687807e-01	5.654237e-01	1.660166e+00	1.845917e+02
market_impact	-3.010645e-05	3.383862e-05	1.309451e-04	8.165527e-03

❶ Appending market impact into the `liq_vol_all` dataframe.

These are the liquidity measures that we take advantage of in the process of modeling the liquidity via GMM. Now let's discuss this via a probabilistic unsupervised learning algorithm.

Gaussian Mixture Model

What happens if we have data with several modes that represent different aspects of the data? Or let's put it in the context of liquidity measures, how can you model liquidity measures with different mean variance? As you can imagine, data consisting of liquidity measures is multimodal, meaning that there exists several different high-probability masses and our task is to find out which model fits best to this type of data.

It is evident that the proposed model is supposed to include a mixture of several components, and without knowing the specific liquidity measure, it should be clustered based on the values obtained from the measures. To recap, we will have one big dataset that includes all liquidity measures, and assuming for the moment that we forgot to assign labels to these measures, we need a model that presents different distributions of these measures without knowing the labels. This model is GMM, which enables us to model multimodal data without knowing the names of the variables.

Considering the different focus of the liquidity measures introduced before, if we somehow manage to model this data, that means we can capture different liquidity level at different times. For instance, liquidity in a high-volatility period cannot be modeled in the same way as a low-volatility period. In a similar vein, given the depth

of the market, we need to focus on these different aspects of liquidity. GMM provides us with a tool to address this problem.

Long story short, if a market is experiencing a boom period, which coincides with high volatility, volume and transaction cost-based measures would be good choices, and if a market ends up with price discontinuity, price-based measures would be the optimal choice. Of course, we are not talking about one-size-fits-all measures—there may be some instances in which a mixture of measures would work better.

As put by VanderPlas (2016), for K-means to succeed, cluster models must have circular characteristics. Nevertheless, many financial variables exhibit non-circular shapes that make it hard to model via K-means. As is readily observable, liquidity measures overlap and do not have circular shapes, so GMM with its probabilistic nature would be a good choice for modeling this type of data, as described by Fraley and Raftery (1998):

One advantage of the mixture-model approach to clustering is that it allows the use of approximate Bayes factors to compare models. This gives a systematic means of selecting not only the parameterization of the model (and hence the clustering method), but also the number of clusters.

In this part, we would like to import necessary libraries to be used in the GMM. Also, scaling is applied, which is an essential step in clustering as we have mixed numerical values in the dataframe. In the last part of the code that follows, a histogram is drawn to observe the multimodality in the data (Figure 1). This is a phenomenon that we have discussed in the very first part of this section.

```
In [28]: from sklearn.mixture import GaussianMixture
         from sklearn.preprocessing import StandardScaler

In [29]: liq_measures_all2 = liq_measures_all.dropna()
         scaled_liq = StandardScaler().fit_transform(liq_measures_all2)

In [30]: kwargs = dict(alpha=0.5, bins=50, stacked=True)
         plt.hist(liq_measures_all.loc[:, 'percent_quoted_ba'],
                  **kwargs, label='TC-based')
         plt.hist(liq_measures_all.loc[:, 'turnover_ratio'],
                  **kwargs, label='Volume-based')
         plt.hist(liq_measures_all.loc[:, 'market_impact'],
                  **kwargs, label='Market-based')
         plt.title('Multi Modality of the Liquidity Measures')
         plt.legend()
         plt.show()
```

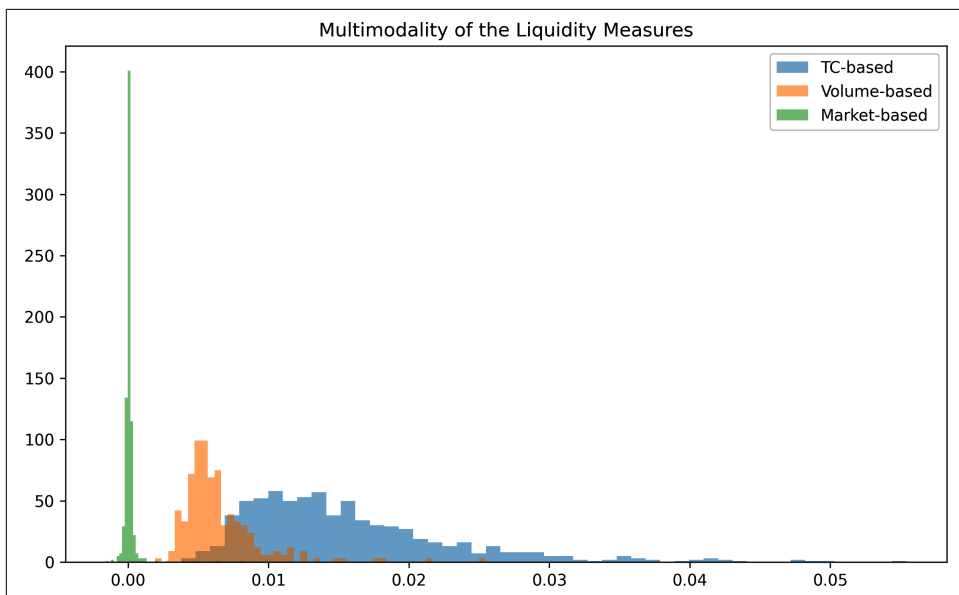


Figure 1. Multimodality of the liquidity measures

And now, given the transaction cost, volume, and market-based liquidity measures, multimodality (i.e., three peaks) can be easily observed in Figure 1. Due to the scaling issue, the price impact-based liquidity dimension is not included in the histogram.

Now, let's run GMM and see how we can cluster the liquidity measures. But first, a common question arises: how many clusters should we have? To address this question, we'll use BIC again, and generate the plot shown in Figure 2:

```
In [31]: n_components = np.arange(1, 10)
         clusters = [GaussianMixture(n, covariance_type='spherical',
                                     random_state=0).fit(scaled_liq)
                     for n in n_components] ❶
         plt.plot(n_components, [m.bic(scaled_liq) for m in clusters]) ❷
         plt.title('Optimum Number of Components')
         plt.xlabel('n_components')
         plt.ylabel('BIC values')
         plt.show()
```

- ❶ Generating different BIC values based on different numbers of clusters
- ❷ Drawing a line plot for BIC values given number of components

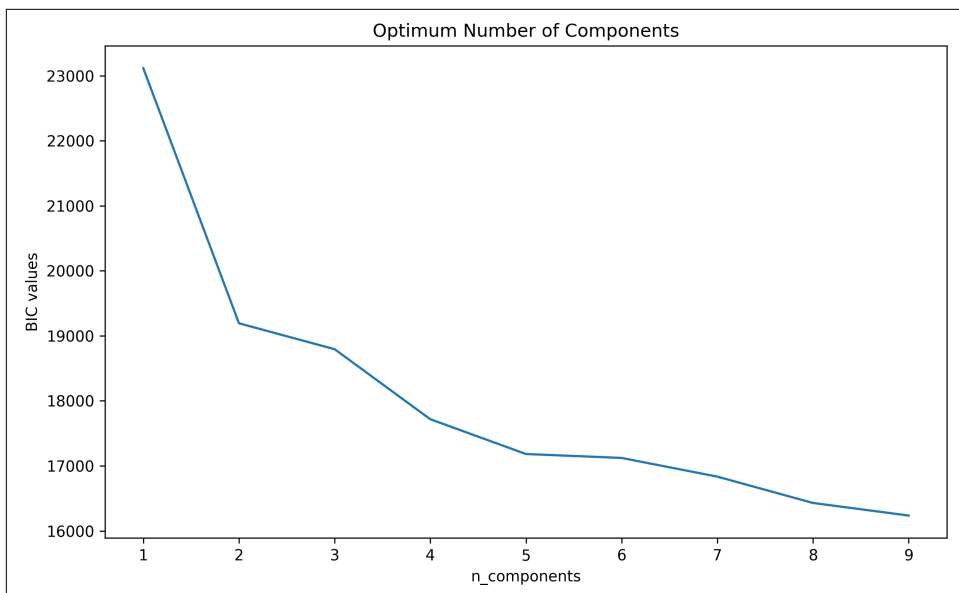


Figure 2. Optimum number of components

Figure 2 shows us that the line seems to flatten out after the third cluster, making that an ideal point at which to stop.

Using the following code, we are able to detect the state by which data is best represented. The term *state* represents nothing but the cluster with the highest posterior probability. It means that this specific state accounts for the dynamics of the data most. In this case, State-3 with a probability of 0.55 is the most likely state to explain the dynamics of the data:

```
In [32]: def cluster_state(data, nstates):
    gmm = GaussianMixture(n_components=nstates,
                           covariance_type='spherical',
                           init_params='kmeans') ❶
    gmm_fit = gmm.fit(scaled_liq) ❷
    labels = gmm_fit.predict(scaled_liq) ❸
    state_probs = gmm.predict_proba(scaled_liq) ❹
    state_probs_df = pd.DataFrame(state_probs,
                                   columns=['state-1', 'state-2',
                                           'state-3'])
    state_prob_means = [state_probs_df.iloc[:, i].mean()
                        for i in range(len(state_probs_df.columns))] ❺
    if np.max(state_prob_means) == state_prob_means[0]:
        print('State-1 is likely to occur with a probability of {:.4f}'
              .format(state_prob_means[0]))
    elif np.max(state_prob_means) == state_prob_means[1]:
        print('State-2 is likely to occur with a probability of {:.4f}'
              .format(state_prob_means[1]))
```

```

    else:
        print('State-3 is likely to occur with a probability of {:.4f}'
              .format(state_prob_means[2]))
    return state_probs

In [33]: state_probs = cluster_state(scaled_liq, 3)
        print(f'State probabilities are {state_probs.mean(axis=0)}')

        State-3 is likely to occur with a probability of 0.550297
        State probabilities are [0.06285593 0.38684657 0.5502975 ]

```

- ❶ Configuring the GMM
- ❷ Fitting GMM with scaled data
- ❸ Running prediction
- ❹ Obtaining the state probabilities
- ❺ Computing the average of all three state probabilities

All right, does it not make sense to apply GMM to cluster liquidity measures and extract the likely state to represent it as one-dimensional data? It literally makes our lives easier because at the end of the data, we come up with only one cluster with highest probability. But what would you think if we applied PCA to fully understand which variables are correlated with the prevailing state? In PCA, we are able to build a bridge between components and features using loadings so that we can analyze which liquidity measures have the defining characteristics of a specific period.

As a first step, let's apply PCA and create a scree plot (Figure 3) to determine the number of components we are working with:

```

In [34]: from sklearn.decomposition import PCA

In [35]: pca = PCA(n_components=11)
        components = pca.fit_transform(scaled_liq)
        plt.plot(pca.explained_variance_ratio_)
        plt.title('Scree Plot')
        plt.xlabel('Number of Components')
        plt.ylabel('% of Explained Variance')
        plt.show()

```

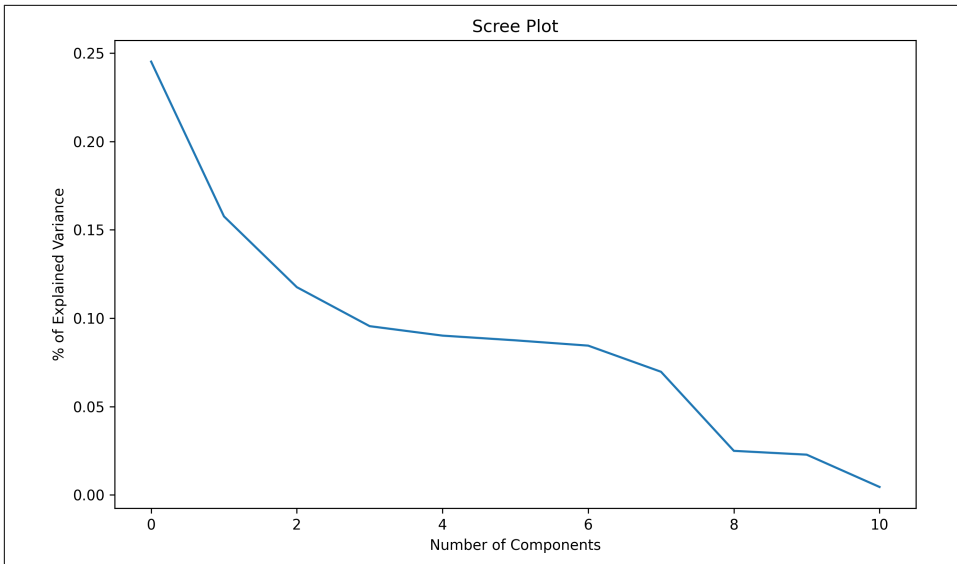


Figure 3. Scree plot

Based on Figure 3, we'll decide to stop at component 3.

As we now have determined the number of components, let's rerun PCA with three components and GMM. Similar to our previous GMM application, posterior probability is calculated and assigned to a variable named `state_probs`:

```
In [36]: def gmm_pca(data, nstate):
          pca = PCA(n_components=3)
          components = pca.fit_transform(data)
          mxtd = GaussianMixture(n_components=nstate,
                                covariance_type='spherical')
          gmm = mxtd.fit(components)
          labels = gmm.predict(components)
          state_probs = gmm.predict_proba(components)
          return state_probs, pca

In [37]: state_probs, pca = gmm_pca(scaled_liq, 3)
          print(f'State probabilities are {state_probs.mean(axis=0)}')
          State probabilities are [0.7329713  0.25076855 0.01626015]
```

In what follows, we find out the state with the highest probability, and it turns out to be State-1 with a probability of 73%:

```
In [38]: def wpc():
          state_probs_df = pd.DataFrame(state_probs,
                                       columns=['state-1', 'state-2',
                                                'state-3'])
          state_prob_means = [state_probs_df.iloc[:, i].mean()
                              for i in range(len(state_probs_df.columns))]
```

```

if np.max(state_prob_means) == state_prob_means[0]:
    print('State-1 is likely to occur with a probability of {:.4f}'
          .format(state_prob_means[0]))
elif np.max(state_prob_means) == state_prob_means[1]:
    print('State-2 is likely to occur with a probability of {:.4f}'
          .format(state_prob_means[1]))
else:
    print('State-3 is likely to occur with a probability of {:.4f}'
          .format(state_prob_means[2]))

wpc()
State-1 is likely to occur with a probability of 0.732971

```

Let's now turn our attention to finding which liquidity measures matter most using loading analysis. This analysis suggests that turnover_ratio, percent_quoted_ba, percent_effective_ba, amihud, and florackis ratios are the liquidity ratios composing the State-1. The following code shows the result:

```

In [39]: loadings = pca.components_.T * np.sqrt(pca.explained_variance_) ❶
         loading_matrix = pd.DataFrame(loadings,
                                       columns=['PC1', 'PC2', 'PC3'],
                                       index=liq_measures_all.columns)

         loading_matrix
Out[39]:
         PC1      PC2      PC3
liq_ratio    0.116701 -0.115791 -0.196355
Lhh          -0.211827  0.882007 -0.125890
turnover_ratio  0.601041 -0.006381  0.016222
percent_quoted_ba  0.713239  0.140103  0.551385
percent_effective_ba  0.641527  0.154973  0.526933
roll         -0.070192  0.886080 -0.093126
CS_spread     0.013373 -0.299229 -0.092705
amihud        0.849614 -0.020623 -0.488324
florackis     0.710818  0.081948 -0.589693
cet          -0.035736  0.101575  0.001595
market_impact  0.357031  0.095045  0.235266

```

❶ Calculating loading from PCA

Gaussian Mixture Copula Model

Given the complexity and sophistication of financial markets, it is not possible to suggest one-size-fits-all risk models. Thus, financial institutions develop their own models for credit, liquidity, market, and operational risks so that they can manage the risks they face more efficiently and realistically. However, one of the biggest challenges that these financial institutions come across is the correlation, also known as joint distribution, of the risk, as put by Rachev and Stein (2009):

With the emergence of the sub-prime crisis and the following credit crunch, academics, practitioners, philosophers and journalists started searching for causes and failures that led to the turmoil and (almost) unprecedented market deteriorations... the arguments against several methods and models used at Wall Street and throughout the

world are, in many cases, putting those in the wrong light. Beyond the fact that risks and issues were clouded by the securitization, tranching and packaging of underlyings in the credit markets as well as by the unfortunate and somehow misleading role of rating agencies, mathematical models were used in the markets which are now under fire due to their incapability of capturing risks in extreme market phases.

The task of modeling “extreme market phases” leads us to the concept of joint distribution by which we are allowed to model multiple risks with a single distribution.

A model disregarding the interaction of risks is destined for failure. In this respect, an intuitive yet simple approach is proposed: *copulas*.

Copula is a function that maps marginal distribution of individual risks to multivariate distribution, resulting in a joint distribution of many standard uniform random variables. If we are working with a known distribution, such as normal distribution, it is easy to model joint distribution of variables, known as bivariate normal. However, the challenge here is to define the correlation structure between these two variables, and this is the point at which copulas come in (Hull 2012).

With Sklar’s theorem, let F be a marginal continuous cumulative distribution function (CDF) of X^i . A CDF transformation maps a random variable to a scalar that is uniformly distributed in $[0,1]$. However, the joint distribution of all these marginal CDFs does not follow uniform distribution and a copula function (Kasa and Rajan 2020):

$$C: [0, 1]^i \rightarrow [0, 1]$$

where i shows the number of marginal CDFs. In other words, in the bivariate case, i takes the value of 2 and the function becomes:

$$C: [0, 1]^2 \rightarrow [0, 1]$$

Hence:

$$F(x_1, x_2) \equiv C(F_1(x_1), \dots, F_i(x_i))$$

where C is copula and unique given the marginal distribution of F_i s are continuous and F is joint cumulative distribution.

Alternatively, the copula function can be described by individual marginal densities:

$$f(x) = C(F_1(x_1), \dots, F_i(x_i)) \prod_{j=1}^i f_j(x_j)$$

where $f(x)$ denotes multivariate density, and f_j is marginal density of the j^{th} asset.

We cannot complete our discussion without stating the assumptions that we need to satisfy for copulas. Here are the assumptions from Bouye (2000):

1. $C = S_1 \times S_2$, where S_1 and S_2 are non-empty subsets of $[0,1]$.
2. C is an increasing function such that $0 \leq u_1 \leq u_2 \leq 1$ and $0 \leq v_1 \leq v_2 \leq 1$.

$$C([u_1, v_1] \times [u_2, v_2]) \equiv C(u_2, v_2) - C(u_2, v_1) - C(u_1, v_2) + C(u_1, v_1) \geq 0$$

3. For every u in S_1 and for every v in S_2 : $C(u, 1) = u$ and $C(1, v) = v$.

After a long theoretical discussion about copulas, you may be tempted to think about the complexity of its coding in Python. No worries, we have a library for that and it is really easy to apply. The name of the Python library for copulas is called Copulae, and we will make use of it in the following:

```
In [40]: from copulae.mixtures.gmc.gmc import GaussianMixtureCopula ❶

In [41]: _, dim = scaled_liq.shape
          gmcm = GaussianMixtureCopula(n_clusters=3, ndim=dim) ❷

In [42]: gmcm_fit = gmcm.fit(scaled_liq, method='kmeans',
                             criteria='GMCM', eps=0.0001) ❸
          state_prob = gmcm_fit.params.prob
          print(f'The state {np.argmax(state_prob) + 1} is likely to occur')
          print(f'State probabilities based on GMCM are {state_prob}')
          The state 2 is likely to occur
          State probabilities based on GMCM are [0.3197832  0.34146341 0.
          33875339]
```

- ❶ Importing GaussianMixtureCopula from copulae
- ❷ Configuring GMCM with the number of clusters and dimensions
- ❸ Fitting the GMCM

The result suggests that when the correlation is taken into account, State-2 prevails, but the posterior probabilities are very close to each other, implying that when correlation between liquidity measures comes into the picture, commonality in liquidity stands out.

Conclusion

Liquidity risk has been under a microscope for over a decade as it is an important source of risk by itself and also has high correlation with other financial risks.

This chapter introduces a new method for liquidity modeling based on GMM, which allows us to model multivariate data and generate clusters. Given the posterior probability of these clusters, we were able to determine which cluster represented the defining characteristics of the data. However, without considering the correlation structure of the liquidity measures, our model may not have been a good representation of reality. Thus, to address this concern, we introduced GMCM, and the defining cluster was redefined by taking into account the correlation structure among the variables.

After completing the liquidity modeling, we are now ready to discuss another important source of financial risk: *operational risk*. Operational risk may arise for a variety of reasons, but we will discuss operational risk via fraudulent activities.

References

Articles cited in this chapter:

- Abdi, Farshid, and Angelo Ranaldo. 2017. "A Simple Estimation of Bid-Ask Spreads from Daily Close, High, and Low Prices." *The Review of Financial Studies* 30 (12): 4437-4480.
- Baker, H. Kent, and Halil Kiyamaz, eds. 2013. *Market Microstructure in Emerging and Developed Markets: Price Discovery, Information Flows, and Transaction Costs*. Hoboken, New Jersey: John Wiley and Sons.
- Bessembinder, Hendrik, and Kumar Venkataraman. 2010. "Bid-Ask Spreads." in *Encyclopedia of Quantitative Finance*, edited b. Rama Cont. Hoboken, NJ: John Wiley and Sons.
- Blume, Lawrence, David Easley, and Maureen O'Hara. 1994 "Market Statistics and Technical Analysis: The Role of Volume." *The Journal of Finance* 49 (1): 153-181.
- Bouyé, Eric, Valdo Durrleman, Ashkan Nikeghbali, Gaël Riboulet, and Thierry Roncalli. 2000. "Copulas for Finance: A Reading Guide and Some Applications." Available at SSRN 1032533.
- Chuck, Prince. 2007. "Citigroup Chief Stays Bullish on Buy-Outs." *Financial Times*. <https://oreil.ly/nKOZk>.
- Corwin, Shane A., and Paul Schultz. 2012. "A Simple Way to Estimate Bid-Ask Spreads from Daily High and Low Prices." *The Journal of Finance* 67 (2): 719-760.
-

- Florackis, Chris, Andros Gregoriou, and Alexandros Kostakis. 2011. "Trading Frequency and Asset Pricing on the London Stock Exchange: Evidence from a New Price Impact Ratio." *Journal of Banking and Finance* 35 (12): 3335-3350.
- Fraley, Chris, and Adrian E. Raftery. 1998. "How Many Clusters? Which Clustering Method? Answers via Model-Based Cluster Analysis." *The Computer Journal* 41 (8): 578-588.
- Gabrielsen, Alexandros, Massimiliano Marzo, and Paolo Zagaglia. 2011. "Measuring Market Liquidity: An Introductory Survey." *SRN Electronic Journal*.
- Harris, Lawrence. 1990. "Statistical Properties of the Roll Serial Covariance Bid/Ask Spread Estimator." *The Journal of Finance* 45 (2): 579-590.
- Gaygisiz, Esma, Abdullah Karasan, and Alper Hekimoglu. 2021. "Analyses of factors of Market Microstructure: Price impact, liquidity, and Volatility." *Optimization* (Forthcoming).
- Kasa, Siva Rajesh, and Vaibhav Rajan. 2020. "Improved Inference of Gaussian Mixture Copula Model for Clustering and Reproducibility Analysis using Automatic Differentiation." arXiv preprint arXiv:2010.14359.
- Kyle, Albert S. 1985. "Continuous Auctions and Insider Trading." *Econometrica* 53 (6): 1315-1335.
- Le, Huong, and Andros Gregoriou. 2020. "How Do You Capture Liquidity? A Review of the Literature on Low-Frequency Stock Liquidity." *Journal of Economic Surveys* 34 (5): 1170-1186.
- Lou, Xiaoxia, and Tao Shu. 2017. "Price Impact or Trading Volume: Why Is the Amihud (2002) measure Priced?." *The Review of Financial Studies* 30 (12): 4481-4520.
- Nikolaou, Kleopatra. 2009. "Liquidity (Risk) concepts: Definitions and Interactions." European Central Bank Working Paper Series 1008.
- Rachev, S. T., W. Sun, and M. Stein. 2009. "Copula Concepts in Financial Markets." *Portfolio Institutionell* (4): 12-15.
- Roll, Richard. 1984. "A Simple Implicit Measure of the Effective Bid-Ask Spread in an Efficient Market." *The Journal of Finance* 29 (4): 1127-1139.
- Sarr, Abdourahmane, and Tonny Lybek. 2002. "Measuring liquidity in financial markets." IMF Working Papers (02/232): 1-64.

Books and online sources cited in this chapter:

- Hull, John. 2012. *Risk Management and Financial Institutions*. Hoboken, New Jersey: John Wiley and Sons.
- VanderPlas, Jake. 2016. *Python Data Science Handbook: Essential Tools for Working with Data*. Sebastopol: O'Reilly.
-